

Project - 1

Dynamic Programming & Optimization

I know it's DP but I just can't prove it..

Implement and compare algorithms like Dijkstra, Bellman-Ford, A*, IDA*, and D* for grid navigation (e.g., robot pathfinding). Add constraints like dynamic obstacles (other robots doing the navigation), fuel limits, or terrain costs. Visualise the path with color-coded grids.

Project - 2

Graph Theory

Why did the vertex break up with the edge?

Given a synthetic friendship graph (e.g., Facebook-like network), analyze connected components, centrality, community detection (Union-Find, BFS, DFS, PageRank). Use random "personality tags" and visualize friendship clusters. Can you also implement a recommender system?

Project - 3

Greedy + Divide & Conquer

When is it okay to be greedy?

Implement algorithms to minimize completion time of jobs with dependencies (like CPU scheduling). Compare greedy heuristics vs optimal DP vs branch-and-bound. Do a robust analysis...

Project - 4

Computational Geometry

Animating convex hull construction:

Expectation: smooth elegant movement

Reality: blinking points.

Given a set of 2D points, implement Graham's scan, Jarvis March, and QuickHull. Compute area, perimeter, and test random points for inclusion. Animate hull construction visually.

Project - 5

String & Pattern Matching

Implement and analyze multiple string and pattern matching algorithms on DNA sequences (e.g., E. coli genome), including KMP, Boyer-Moore, Suffix Trees for **exact matching**, and Shift-Or and Levenshtein Distance Search for approximate/**fuzzy matching**. Evaluate their performance on multiple datasets, compare latency and memory usage with Python's built-in regex, and visualize results by highlighting matches or motifs.

Project - 6

P vs NP Demonstrator

The P vs NP question is one of the deepest and most important open problems in all of mathematics and computer science. It asks whether every problem whose solution can be verified quickly (in polynomial time, the class NP) can also be solved quickly (the class P). Learn more how this problem really is the law/rule of the universe itself [in this video](#) and [in this short paper](#). For more curious ones out there [this](#) might be helpful.

Discovery of NP-complete problems was one of the biggest revolutions in computability theory. Pick 3–4 **NP-complete** problems (e.g. SAT, 3SAT, Subset Sum, Hamiltonian Path, Vertex Cover) and show how each reduces to another. Implement both the reduction and a brute-force solver for small cases. Learn more about NP-complete problems [here](#) and [here](#).

Project - 7

Approximation Algorithms in Practice

NP-Hard is a s-class of problems which are at least as hard as the hardest problems in NP. Problems that are NP-hard do not have to be elements of NP; indeed, they may not even be decidable.

Take some classic **NP-hard** problems such as Traveling Salesman Problem (TSP), Vertex Cover, Set Cover etc.. and implement them for Exact exponential solver (for small $n \leq 15$) and Polynomial-time approximation algorithm. Compare solution quality and runtime growth.

Project - 8

Randomized Algorithms and Complexity

Randomized algorithms use random numbers as part of their logic to make decisions or improve performance. Instead of always producing the same output or taking the same path, they rely on randomness to achieve good expected performance or correctness.

Implement randomized algorithms such as Randomized QuickSort, Monte Carlo primality test (Miller–Rabin), Randomized MinCut (Karger's Algorithm) and measure expected runtime and error probability empirically. Can you find some patterns?

Project - 9

SAT Solver

*The Boolean Satisfiability (SAT) problem was the first problem proven NP-[complete](#) by Richard Karp in his paper “Reducibility Among Combinatorial Problems” and Stephen Cook's **Cook–Levin Theorem** or **Cook's Theorem**. Learn more about SAT [here](#).*

Implement a simple **DPLL SAT solver** and a backtracking **Sudoku solver**, both being exponential in the worst case. Measure how heuristics (unit propagation, MRV, backjumping) change practical runtimes. Additionally can you also encode Sudoku to CNF (one-hot encoding for cell values + constraints)? Now, SAT exemplifies the elegance and universality of computational hardness because almost any combinatorial problem can be expressed as a SAT instance! Therefore you are invited to explore more!

Project - 10

Graph Coloring

*The Graph Coloring problem is a canonical **NP-hard** challenge in computer science. The goal is to assign colors to a graph's vertices such that no two adjacent vertices share the same color, using the minimum number of colors possible.*

Implement a spectrum of algorithms that explore the core trade-off between a fast solution and an optimal one. Include a simple greedy heuristic (**Welsh-Powell**), a more advanced adaptive heuristic (**DSatur**), and a

metaheuristic capable of finding high-quality solutions (**Simulated Annealing**). For a baseline, also implement an exact, exponential-time solver using **Dynamic Programming** to find the true chromatic number. Measure and compare how solution quality (number of colors used) and practical runtimes differ across these fundamentally distinct approaches.

Project - 11

Maximum Clique

*The Maximum Clique problem is a canonical **NP-hard** challenge in computer science. The goal is to find the largest possible subset of vertices in a graph where every vertex is connected to every other vertex in the subset.*

Implement a spectrum of algorithms that explore the core trade-off between a fast solution and an optimal one. Include a **simple greedy heuristic**, a more advanced **local search or randomized heuristic** to improve upon greedy results, and a metaheuristic like **Simulated Annealing** capable of finding high-quality solutions. For a baseline, also implement an exact, exponential-time solver like the **Bron-Kerbosch algorithm** to find the true maximum clique size. Measure and compare how solution quality (clique size) and practical runtimes differ across these fundamentally distinct approaches.

Project - 12

Maximum Flow & Minimum Cut

Me: optimizes flow

Also me: still can't optimize my sleep schedule 🧠🧠🧠

Implement a spectrum of algorithms that represent the historical evolution of solving this problem. Begin with the classic **Ford-Fulkerson method**. Then, implement the significantly more efficient **Dinic's algorithm**. Finally, tackle the fundamentally different **Push-relabel algorithm**. A core part of the project is to also implement the logic to find the corresponding **Minimum Cut** from the residual graph, empirically demonstrating the max-flow min-cut theorem.

Measure and compare the practical runtimes and scalability of these three major algorithms. The goal is to analyze how graph structure (e.g., density, specific capacity layouts) affects the performance of each approach and to empirically

validate the significant theoretical performance gains offered by the more advanced algorithms over the foundational method.